

CUDA Bitonic Sort: Optimization and Performance Counter Analysis

1. Problem and approach

Bitonic sort is a data-oblivious sorting network: for an array of $N = 2^p$ it executes $p(p+1)/2$ compare-exchange stages, each fully parallel. Every stage at subsequence length k and stride s pairs element i with $i+s$. Input sizes are padded up to the next power of two with a sentinel above the maximum generated value, so the padding sorts to the tail and is discarded on copy-back.

The whole optimization effort follows from one observation about the naive mapping. At 100M elements $\text{pad_size} = 2^{27}$, giving 378 stages. Launching one kernel per stage means 378 round trips through global memory, each reading and writing the entire array: $378 \times 512 \text{ MB read} = \sim 400 \text{ GB}$ of traffic for a 512 MB array. At the H100's 3.35 TB/s that is a $\sim 120 \text{ ms}$ floor, above the 80 ms kernel-time target. Arithmetic per stage is two loads, one compare, two stores, so the kernel is bandwidth-bound and **every optimization below reduces bytes moved or passes taken, not arithmetic.**

2. Optimization 1: shared-memory tiling

A block loads `TILE` contiguous, `TILE`-aligned elements into shared memory. Both members of every pair at stride s fall inside that window exactly when $2s \leq \text{TILE}$, so all such stages need no inter-block communication. This splits the work into two kernels:

- **Kernel A** (`bitonic_merge_small_k`) handles every $k \leq \text{TILE}$. Each such k has all strides $\leq k/2 \leq \text{TILE}/2$, so the entire triangle of stages runs in **one launch**: load tile, loop k and stride with `__syncthreads` between stages, write once.
- **Kernel B** (`bitonic_merge_large_k`) handles $k > \text{TILE}$. Strides $\geq \text{TILE}$ must go through global memory one stage per launch; once the stride falls to $\text{TILE}/2$ the remaining cascade for that k completes in one shared-memory launch.

At `TILE = 16384` this covers all 378 stages in 105 launches instead of 378. Kernel B receives k as an argument because the comparison direction depends on global position, not tile-local position. Since $k \geq 2 \times \text{TILE}$ there, every element of a tile shares one direction, so `dir` is computed **once per launch** rather than per pair.

3. Optimization 2: data type

`main.cu` fills with `rand() % 1000`, so values never exceed 999 and `short` (max 32,767) is a valid `DTYPE`. This halves H2D, D2H, and all kernel traffic at once, measured as an exact 2.0x on both transfers. It also halves shared memory per tile, so `TILE` doubles from 8192 to 16384 within the same 48 KB budget, removing a further 14 global launches. A second effect was unplanned: shared bank conflicts collapsed from 96.0 M to 0.33 M, because two 16-bit elements share one 32-bit bank word, so the low-stride pattern that previously took 2-way conflicts now resolves in a single word.

4. Optimization 3: transfers

D2H initially cost 130 ms. The first hypothesis was lazy page-faulting in `malloc`, so the output buffer was switched to `cudaHostAlloc`; this barely helped, because pinning 400 MB itself costs $\sim 83 \text{ ms}$ and simply replaces the fault cost. The fix is **overlap**: pinning is host-side work and does not block the GPU, so issuing the sort kernels first (asynchronous, returns immediately) and allocating the pinned buffer while the GPU is busy hides the pinning entirely behind kernel time. D2H fell from 130 ms to 3.6 ms.

H2D remains at ~9.6 GB/s because `arrCpu` is `malloc'd` in `main.cu` and every transfer is staged through a driver-internal pinned buffer. `cudaHostRegister` page-locks that buffer in place for ~7.6 ms per 200 MB, which is ~5x cheaper per byte than allocating pinned memory, letting DMA read `arrCpu` directly.

5. Optimization 4: address arithmetic

`ncu` showed kernels A and B at 75% compute throughput but only 25% memory throughput, with `math_pipe_throttle` at ~30% of warp cycles: they were compute-bound, not memory-bound. The cause is `i = (t/stride)*2*stride + (t%stride)` and `dir = ((base+i)/k) % 2`. There is no hardware integer divide, so a variable divisor compiles to a `MUFU.RCP + I2F/F2I + multiply-high` sequence, and shared loads/stores were only 86 of 615 SASS instructions. Both divisors are powers of two, so `2t - (t & (stride-1))` and `(g >> log2k) & 1` are equivalent. This cut `small_k` by 23% of its instructions and kernel time by 16%.

The rewrite pushed registers from 32 to 40, which halved occupancy: at 1024 threads, 32 registers exactly fills the SM's 65,536-register file for two blocks, so 40 admits only one. `__launch_bounds__(1024, 2)` restores 32 registers with zero spilling.

6. Optimization 5: vectorized global merge

The global stage kernel was still latency-bound (long-scoreboard 31%). Loading `int4` (8 shorts per thread) raised sectors per request from 2.00 to 16.00 - each warp instruction now moves 512 B instead of 64 B for the same total bytes. Memory throughput on that kernel rose 30% to 66% and total kernel time fell 44%, from 7.16 ms to 3.98 ms at 10M.

7. Results

All runs on H100, `grade.py`, 100M elements.

Configuration	H2D (ms)	Kernel (ms)	D2H (ms)	meps	Occupancy	Score
Naive, one kernel per stage, <code>int</code>	42.3	123.9	130.3	337	41.8%	13.16
+ shared-memory tiling	40.5	73.5	129.4	409	58.0%	16.70
+ <code>short</code> , TILE 16384, pinned overlap	21.1	61.2	3.6	1163	69.4%	21.00
+ <code>int4</code> vectorized global merge	20.8	62.2	3.6	1146	67.5%	21.00

Net: 337 to 1163 meps, a 3.4x improvement, with D2H down 36x and the kernel down 2.0x.

8. Performance counters

`ncu --set full` with explicit metrics, 10M elements, exported from the `.ncu-rep` (67 launches).

Kernel	Launches	Duration (ms)	% time	SM thr.	Mem thr.	Achieved occ.	Theoretical occ.
<code>bitonic_merge_small_k</code>	1	1.621	40.7%	78.20%	33.06%	93.78%	100.0%
<code>bitonic_merge_vec_int4</code>	55	1.123	28.2%	31.75%	66.35%	73.31%	100.0%
<code>bitonic_merge_large_k</code>	10	1.205	30.3%	78.91%	68.81%	93.64%	100.0%
<code>fill_padding</code>	1	0.034	0.9%	23.70%	9.57%	9.50%	100.0%
Total	67	3.984	100.0%	—	—	—	—

Required counters, summed over all launches:

Kernel	Global LD sectors (M)	Global ST sectors (M)	Sectors/request	Local LD/ST	Divergent branches	Achieved occ.
<code>bitonic_merge_small_k</code>	1.0	1.0	2.00	0	0	93.78%
<code>bitonic_merge_vec_int4</code>	57.7	57.4	16.00	0	0	73.31%
<code>bitonic_merge_large_k</code>	10.5	10.5	2.00	0	0	93.64%
<code>fill_padding</code>	0.0	0.4	—	0	0	9.50%
Total	69.2	69.4	7.38	0	0	—

Local memory traffic is zero, confirming no register spilling, and divergent branches are zero (`thread_inst_executed_per_inst_executed` = 32.00 exactly) because the compare-exchange compiles to predicated selects rather than branches. Stall analysis explains the two kernel classes: the shared kernels are limited by `not_selected` (~37%) and `math_pipe_throttle` (~20-28%), while the vectorized global kernel is `long_scoreboard`-limited at 67% with only 0.94 eligible warps per scheduler, i.e. genuinely DRAM-bound, which is the intended end state.

9. Observations and next steps

Memory Throughput reads only 44% and falls short of the 75% target. This is expected rather than a regression: the shared-memory kernels deliberately avoid DRAM, so their low DRAM utilization is the optimization working. Chasing that number would mean undoing the tiling.

Two further steps were identified but not implemented. First, register blocking in the global stage kernel - having each thread resolve `b` consecutive stride levels in registers before writing back would cut the 55 global launches to roughly 21 at `b` = 4, dropping traffic from 48 GB to 19 GB. Second, `fill_padding` contributes 0.9% of runtime but its 9.50% occupancy drags the unweighted per-kernel average that determines the occupancy score; replacing it with a `cudaMemset` does not help because that still launches an internal kernel that the profiler counts.

References

1. CUDA Runtime API, Memory Management. https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__MEMORY.html
2. How to Optimize Data Transfers in CUDA C/C++. <https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/>
3. Boosting Application Performance with GPU Memory Access Tuning. <https://developer.nvidia.com/blog/boosting-application-performance-with-gpu-memory-access-tuning/>
4. Nsight Compute Profiling Guide. <https://docs.nvidia.com/nsight-compute/ProfilingGuide/index.html>